

Deliverable D6

Work package WP.2

Analysis of the implementation of statistical methods in the quantization process

Vypracoval: Ing. Roman Budjač, PhD.

This report focuses on the analysis of the implementation of statistical methods in the quantization process of neural networks. The approach is based on the framework proposed in the study "Layer-wise Statistical Analysis: A Time-Sensitive Framework for Implementing Statistical Methods in Neural Network Training Sessions" (Budjač et.al. 2025, CSOC 2025), which introduces a different approach to the analysis and optimization of models during their training. The primary goal is to use statistical data, acquired in real time, to make fundamentally informed decisions in the quantization process.

Implementation of statistical methods in the quantization process

The implementation of statistical methods into the quantization process represents a paradigm shift from traditional uniform compression approaches to intelligent, adaptive quantization strategies that optimize compression while preserving model quality through evidence-based decisions. This transformation requires a fundamental rethinking of the way we approach neural network compression, implementing a dynamic system that adapts to the unique characteristics of each model and its individual layers instead of applying fixed quantization schemes.

Characterizing the original weight distributions begins with systematic statistical profiling prior to any quantization operations. This initial phase is critical for understanding the inherent properties of the model and establishing reference points for subsequent degradation monitoring. The process uses advanced statistical analysis implemented using the `scipy.stats` and `numpy` libraries, which provide robust tools for numerical calculations and statistical modeling.

The first step in the characterization process is the extraction and preprocessing of weight distributions from each model layer. This process is implemented in Appendix 1, where the `extract_weight_distributions()` function iterates through all model layers and extracts the weight matrices. This function forms the basis of the entire analysis process, as the quality of the extracted data directly affects the accuracy of all subsequent analyses.

Code 1: Extraction and pre-processing of weights_{python}

```
def extract_weight_distributions(model):  
    weight_distributions = {}  
  
    for i, layer in enumerate(model.layers):  
        if hasattr(layer, 'get_weights') and layer.get_weights():  
            weights = layer.get_weights()[0] # Získanie váhovej matice  
            flat_weights = weights.flatten() # Konverzia na 1D pole  
  
            q75, q25 = np.percentile(flat_weights, [75, 25])
```

```
iqr = q75 - q25
lower_bound = q25 - 1.5 * iqr
upper_bound = q75 + 1.5 * iqr
filtered_weights = flat_weights[(flat_weights >= lower_bound) &
                                (flat_weights <= upper_bound)]

weight_distributions[f'layer_{i}'] = {
    'raw_weights': flat_weights,
    'filtered_weights': filtered_weights,
    'shape': weights.shape
}

return weight_distributions
```

As we see in Code 1, the implementation uses the interquartile range (IQR) method to identify and filter outliers. This approach is based on robust statistical principles, where values lying outside 1.5 times the IQR from the first and third quartiles are considered potential outliers. This technique is particularly suitable for dealing with neural network weight distributions, which often exhibit heavy tails and may contain outliers arising during training. Removing these values is critical to ensure the stability of subsequent statistical analyses, while at the same time preserving most of the relevant data - typically more than 99% of the values for normally distributed data.

Successful extraction and filtering of the weights is followed by a comprehensive statistical characterization through the calculation of distributional moments. This stage provides deep insight into the structure of the weight distributions and is implemented in Appendix 2. The `calculate_distribution_moments()` function computes statistical metrics that characterize the properties of the weights in the model.

Code 2: Calculation of statistical moments

python

```
def calculate_distribution_moments(weights):
    moments = {
        'mean': np.mean(weights),
        'variance': np.var(weights),
        'std': np.std(weights),
        'skewness': stats.skew(weights),
        'kurtosis': stats.kurtosis(weights),
        'median': np.median(weights),
        'mad': stats.median_abs_deviation(weights), # Median Absolute
Deviation
```

```
'range': np.ptp(weights), # Peak-to-peak range
'iqr': stats.iqr(weights), # Interquartile range
}

percentiles = [1, 5, 10, 25, 50, 75, 90, 95, 99]
for p in percentiles:
    moments[f'p{p}'] = np.percentile(weights, p)

return moments
```

In Code 2 we see the implementation of the calculation of key statistical moments. Each of these metrics provides some insight into the distributional properties of the weights. The mean and median inform about the central tendency, while their correlation with each other indicates the symmetry of the distribution. The variance and standard deviation quantify the variability of the weights, which is important for understanding how much information can potentially be lost in quantization. Skewness measures the asymmetry of the distribution - positive values indicate a right-handed tail with potentially large positive weights, while negative values indicate a left-handed tail. The skewness (kurtosis) provides information about the concentration of values around the mean and the thickness of the tails of the distribution. High kurtosis values often indicate the presence of outliers or distributions with heavy tails, which can significantly complicate the quantization process.

The median absolute deviation (MAD) is an alternative to the standard deviation that is less sensitive to outliers. The calculation of percentiles provides a detailed picture of the distribution of values across the entire distribution, allowing the identification of concentrations of weights in specific ranges and thus identifying potential problem areas for quantization.

Another critical component of the characterization is the entropy-based analysis, which provides an information-theoretic view of the weight distributions. An implementation of this analysis is presented in Appendix 3, where the `calculate_distribution_entropy()` function computes multiple entropy measures.

Príloha č. 3: Calculation of entropy rates

```
python
def calculate_distribution_entropy(weights, bins='auto'):

    hist, bin_edges = np.histogram(weights, bins=bins, density=True)
    bin_centers = (bin_edges[:-1] + bin_edges[1:]) / 2

    # Normalizácia histogramu na pravdepodobnostnú distribúciu
```

```
hist_normalized = hist / np.sum(hist)
hist_normalized = hist_normalized[hist_normalized > 0]

entropy_measures = {
    'shannon_entropy': stats.entropy(hist_normalized, base=2),
    'differential_entropy': stats.differential_entropy(weights),
    'bin_count': len(hist),
    'effective_bins': np.sum(hist_normalized > 0.01),
}

return entropy_measures, hist, bin_edges
```

As can be seen in Code 3, the entropy calculation starts by generating a histogram with the number of bins automatically determined. The parameter `bins='auto'` uses heuristics implemented in NumPy that select the optimal number of bins based on the characteristics of the data. Normalizing the histogram to a probability distribution is critical for the correct computation of the Shannon entropy, which quantifies the information content of the distribution in bins. Higher entropy values indicate a more uniform distribution of weights across the entire range of values, while lower values indicate a concentration of weights around specific values. This information is key to predicting the impact of quantization - low entropy distributions can often be quantized more aggressively without significant loss of quality.

Differential entropy extends the concept of entropy to continuous distributions and provides a complementary view of information content. The metric of effective baskets, defined as the number of baskets containing more than 1% of the total probability, provides a practical measure of distribution concentration that is more intuitive than entropy alone.

The generation of reference histograms represents a key milestone in the characterization process because it establishes a standardized basis for all subsequent comparisons. The implementation in Appendix 4 demonstrates a sophisticated approach to creating consistent histogram representations across all layers of the model.

Code 4: Generation of reference histograms

python

```
def generate_reference_histograms(weight_distributions, global_bins=100):
    reference_data = {}

    all_weights = np.concatenate([data['raw_weights']
                                   for data in
weight_distributions.values()])
    global_min, global_max = np.min(all_weights), np.max(all_weights)
```

```
# Vytvorenie konzistentných hraníc košov pre všetky vrstvy
bin_edges = np.linspace(global_min, global_max, global_bins + 1)

for layer_name, data in weight_distributions.items():
    weights = data['filtered_weights']

    # Generovanie histogramu s konzistentným binningom
    hist, _ = np.histogram(weights, bins=bin_edges, density=True)
    hist_normalized = hist / np.sum(hist)

    reference_data[layer_name] = {
        'histogram': hist_normalized,
        'bin_edges': bin_edges,
        'bin_centers': (bin_edges[:-1] + bin_edges[1:]) / 2,
        'statistical_moments': calculate_distribution_moments(weights),
        'entropy_measures': calculate_distribution_entropy(weights)[0]
    }

return reference_data
```

In Code 4, we see an implementation that first computes the global range of values across all layers by concatenating all weight vectors into one continuous vector. This approach ensures that all layers will have histograms with identical interval boundaries, which is necessary to allow direct comparisons between layers. The use of `np.linspace()` guarantees a uniform distribution of classes across the entire range of values. A histogram with these global boundaries is then generated for each stratum and normalized to a probability distribution. The resulting data structure contains not only the histograms themselves, but also all the supporting information needed for subsequent analyses, including statistical moments and entropy measures computed using the functions from the previous appendices.

The final step in the basic characterization is to determine the sensitivity profiles of each layer to quantization. This process, implemented in Code 5, provides a quantitative assessment of how different layers respond to different levels of quantization.

Code 5: Determination of sensitivity profiles

```
python

def establish_sensitivity_profiles(model, reference_data,
    weight_distributions,
```

```

        test_bitwidths=[16, 8, , 4]):

    sensitivity_profiles = {}

    for layer_name, ref_data in reference_data.items():
        layer_sensitivity = {}
        original_hist = ref_data['histogram']

        for bitwidth in test_bitwidths:
            original_weights =
weight_distributions[layer_name]['raw_weights']
            quantized_weights =
apply_uniform_quantization(original_weights, bitwidth)

            quant_hist, _ = np.histogram(quantized_weights,
                                         bins=ref_data['bin_edges'],
                                         density=True)
            quant_hist_normalized = quant_hist / np.sum(quant_hist)

            kl_div = stats.entropy(original_hist + 1e-10,
quant_hist_normalized + 1e-10)
            js_div = calculate_js_divergence(original_hist,
quant_hist_normalized)

            layer_sensitivity[bitwidth] = {
                'kl_divergence': kl_div,
                'js_divergence': js_div,
                'sensitivity_score': (kl_div + js_div) / 2
            }

        sensitivity_profiles[layer_name] = layer_sensitivity

    return sensitivity_profiles

```

The implementation in Appendix 5 demonstrates a systematic approach to sensitivity assessment. For each layer and each test bit width (typically 16, 8 and 4 bits), a uniform quantization is applied and the resulting change in distribution is measured. The use of identical bin boundaries from the reference histograms ensures a consistent comparison. Calculating the KL divergence using `stats.entropy()` with the addition of a small epsilon value ($1e-10$) avoids numerical problems when working with zero probabilities. The JS divergence provides a symmetric measure of the

difference between the distributions, which is useful for holistically assessing the impact of quantization. The combined sensitivity score, computed as the average of the KL and JS divergence, provides a uniform metric for comparing the sensitivity of different strata and facilitates decision making on optimal quantization strategies.

Real-time statistical feedback mechanism

The real-time statistical feedback mechanism is the core of the adaptive quantization system. This continuous monitoring system transforms a static quantization process into a dynamic, adaptive system capable of responding to changes in weight distributions during quantization. The implementation uses a parallel architecture that allows simultaneous monitoring of the original and quantized distributions. The basis of this architecture is the `ParallelDistributionTracker` class, implemented in Code 6, which maintains a sophisticated double-buffer system. This design allows independent tracking of the two distributions while maintaining their synchronization and consistency.

Code 6: Parallel tracking of distributions^{python}

```
class ParallelDistributionTracker:
    def __init__(self, reference_data):
        self.reference_data = reference_data
        self.current_distributions = {}
        self.quantized_distributions = {}

    def update_distributions(self, layer_name, original_weights,
                           quantized_weights):

        bin_edges = self.reference_data[layer_name]['bin_edges']

        orig_hist, _ = np.histogram(original_weights, bins=bin_edges,
                                     density=True)
        quant_hist, _ = np.histogram(quantized_weights, bins=bin_edges,
                                     density=True)

        self.current_distributions[layer_name] = orig_hist /
        np.sum(orig_hist)
        self.quantized_distributions[layer_name] = quant_hist /
        np.sum(quant_hist)
```

As we see in Appendix 6, the `ParallelDistributionTracker` class initializes with the reference data obtained during baseline characterization. This architecture ensures that all subsequent comparisons use consistent metrics. The `update_distributions()` method is a key mechanism for keeping distributions up-to-date. At each update, new histograms are generated for both distributions using identical bin boundaries from the

reference data. This approach eliminates the need for additional transformations when comparing distributions and minimizes the computational burden. Normalization of histograms to probability distributions is critical for the correct calculation of divergence rates and ensures that results are interpretable and comparable across different strata and time points.

The calculation of real-time statistical metrics is implemented in Code 7, function `calculate_real_time_divergences()`.

Annex 7: Calculation of divergence rates in real time

python

```
def calculate_real_time_divergences(original_hist, quantized_hist,
epsilon=1e-10):
    """Výpočet divergenčných meraní s numerickou stabilitou"""

    orig_stable = original_hist + epsilon
    quant_stable = quantized_hist + epsilon

    orig_stable = orig_stable / np.sum(orig_stable)
    quant_stable = quant_stable / np.sum(quant_stable)

    divergences = {
        'kl_divergence': stats.entropy(orig_stable, quant_stable),
        'js_divergence': calculate_js_divergence_stable(orig_stable,
quant_stable),
        'cosine_similarity': np.dot(orig_stable, quant_stable) /
(np.linalg.norm(orig_stable) *
np.linalg.norm(quant_stable)),
        'wasserstein_distance': stats.wasserstein_distance(orig_stable,
quant_stable)
    }

    return divergences

def calculate_js_divergence_stable(p, q, epsilon=1e-10):

    m = 0.5 * (p + q)

    js_div = 0.5 * (stats.entropy(p, m) + stats.entropy(q, m))
```

```
return js_div
```

In Code 7 we see the implementation of the calculation of divergence rates. Adding epsilon values to all histogram elements before computing divergence rates is essential to avoid numerical instabilities that would arise from logarithmizing zero values. The epsilon value ($1e-10$) is chosen small enough not to affect the results significantly, but large enough to ensure numerical stability even when working with float32 precision. The renormalization after adding epsilon preserves the properties of the probability distribution.

The function computes selected metrics: the KL divergence for asymmetric information loss measurements, the JS divergence for symmetric comparison of distributions, the cosine similarity for angular closeness of distribution vectors, and the Wasserstein distance for measuring the "transport cost" between distributions. Each of these metrics provides insight into changes in the distributions, and combining them allows for a robust assessment of the impact of quantization.

Conclusion:

The implementation of statistical methods in the quantization process of neural networks represents a significant advance in the optimization of deep learning models. The proposed approach based on layer-wise statistical analysis enables a transition from traditional uniform quantization techniques to adaptive strategies that use advanced statistical analyses to optimize compression while preserving model quality. The present implementation demonstrates that the integration of advanced statistical analyses directly into the quantization process is technically feasible. The developed framework provides a basis for further research in WP.3 and WP.4 work packages.